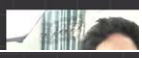
Lec-17

1. — single flag ✗
2. — Peterson's soln 2 threads
3. — Locks / Mutex

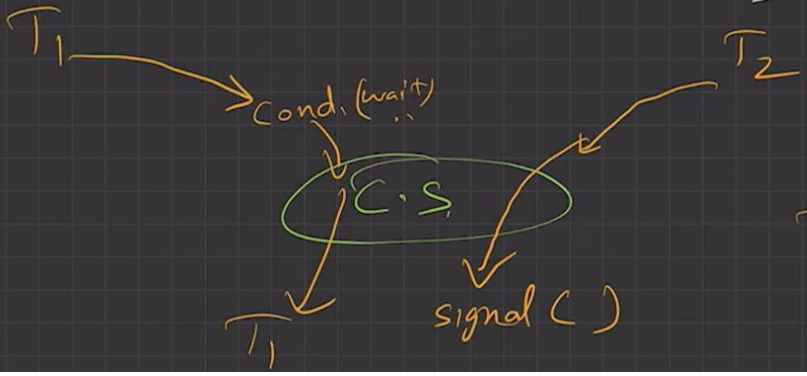
Problems → Busy waiting



* Conditional Variables :-

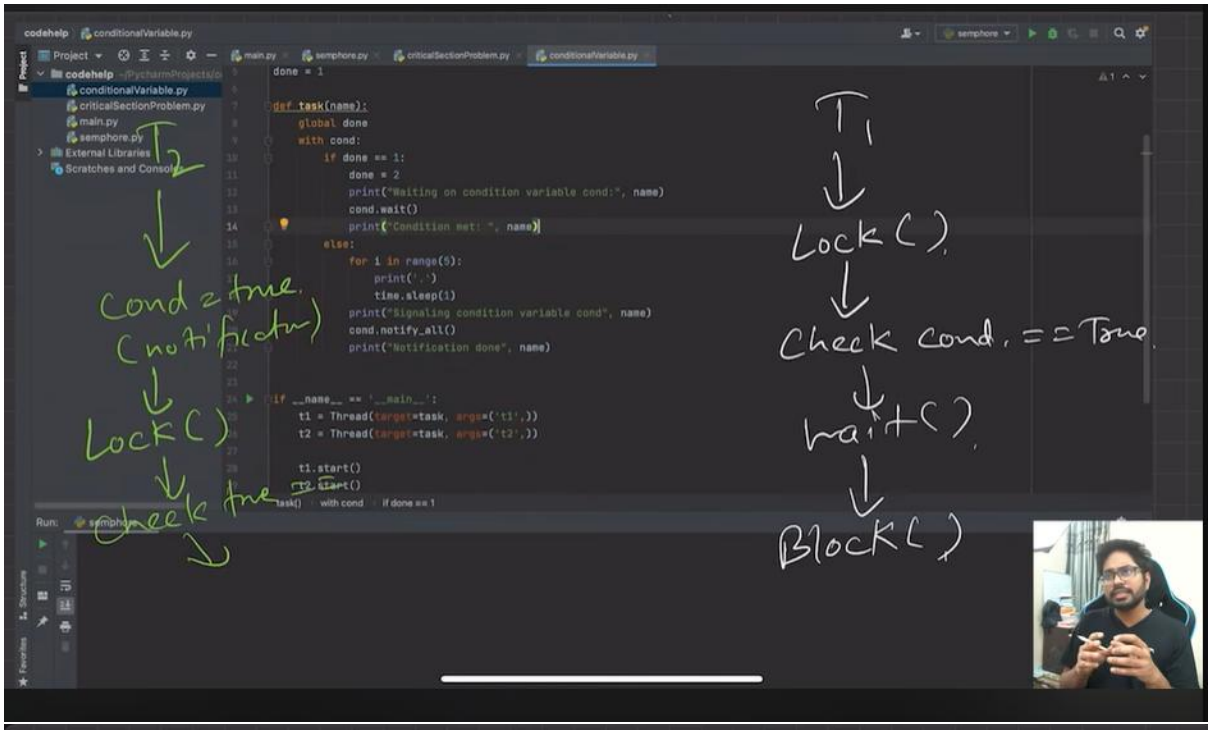
var => cond (lock)

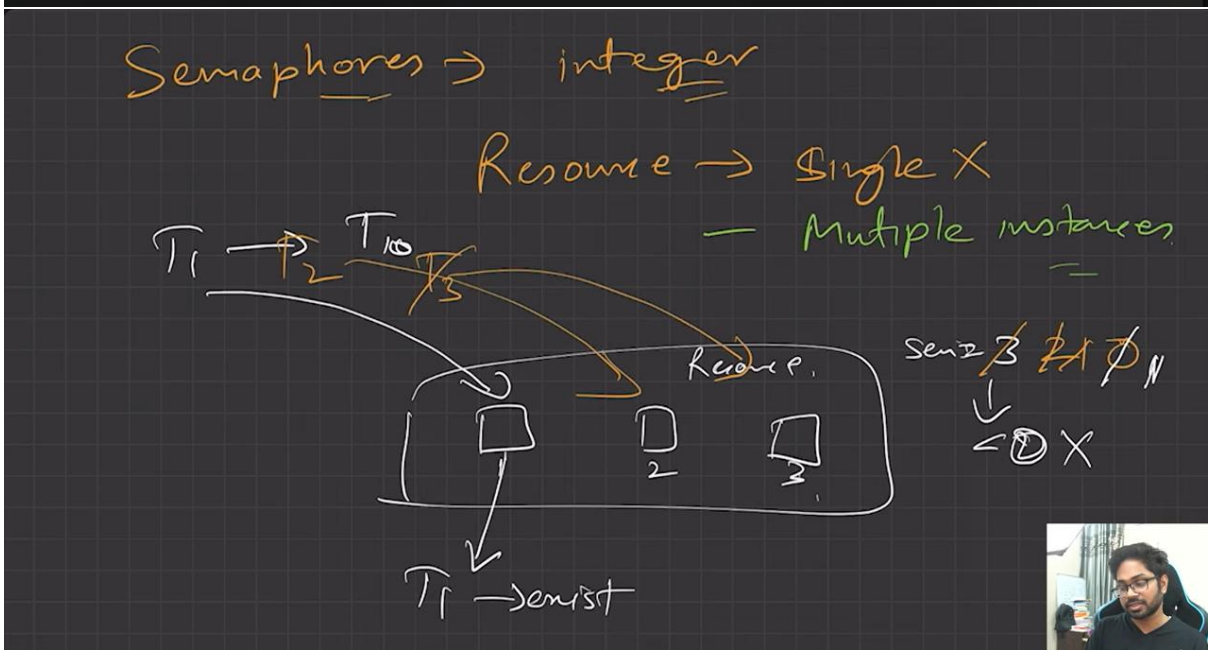
wait signal

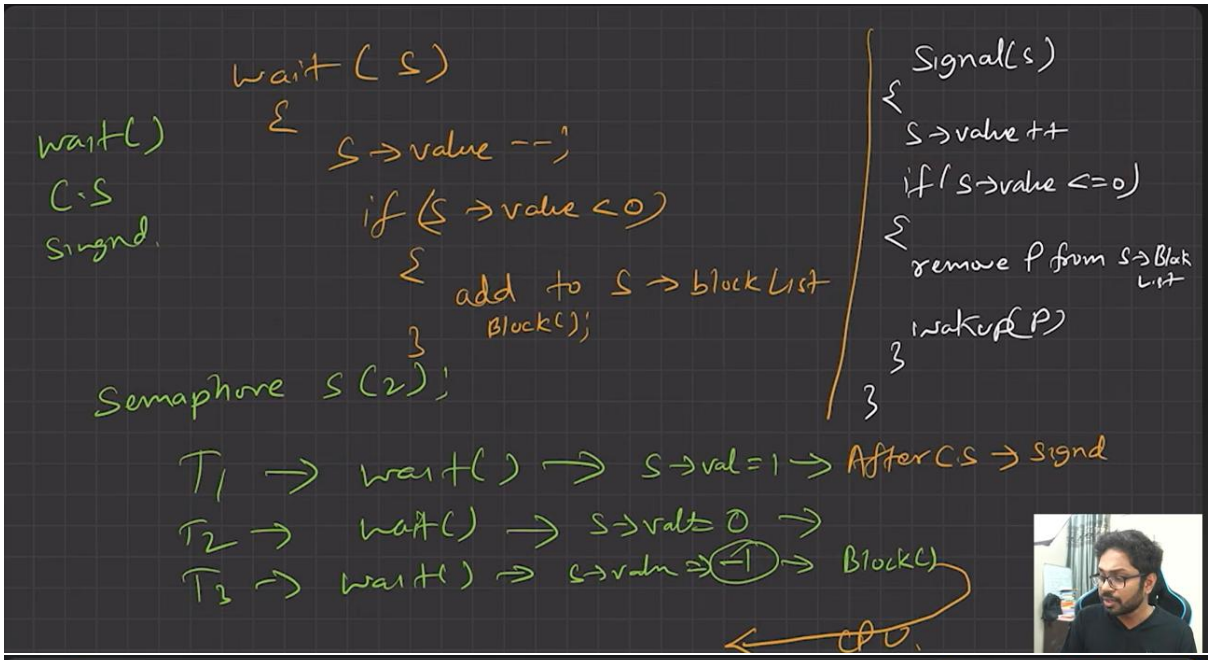


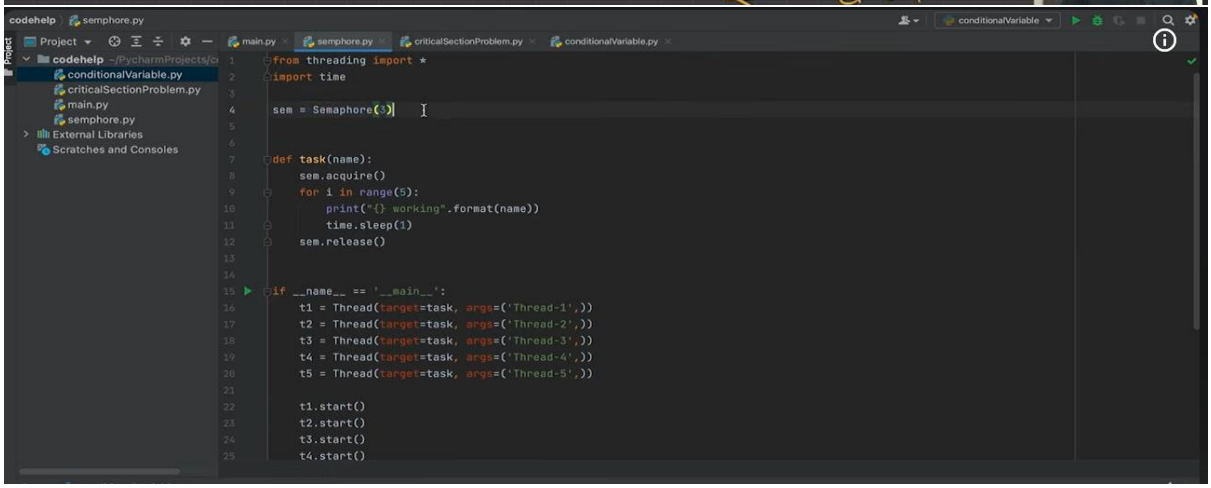
$T_2 \rightarrow T$




codehelp
conditionalVariable.py
Project
main.py
semphore.py
criticalSectionProblem.py
conditionalVariable.py
codehelp
conditionalVariable.py
criticalSectionProblem.py
main.py
semphore.py
External Libraries
Scratches and Consoles
done = 1
def task(name):
global done
with cond:
if done == 1:
done = 2
print("Waiting on condition variable cond:", name)
cond.wait()
print("Condition met: ", name)
else:
for i in range(5):
print('.')
time.sleep(1)
print("Signaling condition variable cond", name)
cond.notify_all()
print("Notification done", name)
if __name__ == '__main__':
t1 = Thread(target=task, args=('t1',))
t2 = Thread(target=task, args=('t2',))
t1.start()
t2.start()
Run:
T2
cond = true
(notification)
Lock()
Check true
T1
Lock()
Check cond. == True
wait()
Block()


Semaphores → integer
Resource → Single X
— Mutiple instances
T1
T2
T10
T3
Resource
1
2
3
Semz 3 2 1 0
<0 X
T1 → Semist


wait(S)
{
S->value--;
if(S->value<0)
{
add to S->blockList
Block();
}
}
wait()
C.S
Signal.
Semaphore S(2);
Signal(S)
{
S->value++
if(S->value<=0)
{
remove P from S->BlockList
wakeup(P)
}
}
T1 -> wait() -> S->val=1 -> After C.S -> Signal
T2 -> wait() -> S->val=0 ->
T3 -> wait() -> S->val=-1 -> Block()


codehelp
semphore.py
conditionalVariable
Project
main.py
semphore.py
criticalSectionProblem.py
conditionalVariable.py
codehelp ~/PycharmProjects/c
conditionalVariable.py
criticalSectionProblem.py
main.py
semphore.py
External Libraries
Scratches and Consoles
from threading import *
import time
sem = Semaphore(3)
def task(name):
sem.acquire()
for i in range(5):
print("{} working".format(name))
time.sleep(1)
sem.release()
if __name__ == '__main__':
t1 = Thread(target=task, args=('Thread-1',))
t2 = Thread(target=task, args=('Thread-2',))
t3 = Thread(target=task, args=('Thread-3',))
t4 = Thread(target=task, args=('Thread-4',))
t5 = Thread(target=task, args=('Thread-5',))
t1.start()
t2.start()
t3.start()
t4.start()